

src port	dst port
length	checksum

Computer Networks

Week 3 — Detailed Study Notes

The Transport Layer · Ports, Sockets & Demultiplexing · UDP · The Internet Checksum

IIT Madras BS — Prof. Ayon Chakraborty

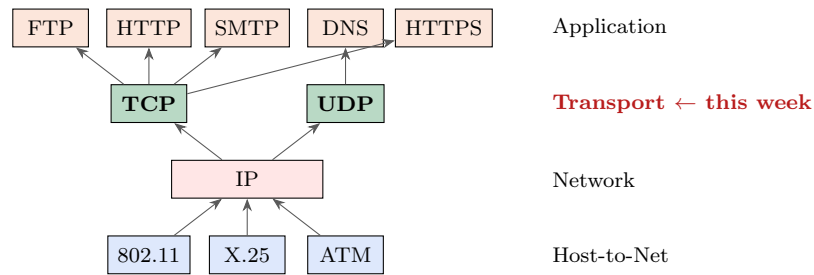
How to read these notes. Every slide concept from the three Week-3 lectures appears here, in slide order, grouped by module (Module 3. n = Lecture n). Box types: **Definition**, **Intuition**, **Formula**, **Common pitfall**, **Beyond the slides** (flagged extra material), grey **What the slide says**. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 3.1 — The Transport Layer	2
1.1	Where transport sits	2
1.2	Transport services and protocols	2
1.3	Transport vs network layer	3
1.4	Identifying a single conversation: the 4-tuple	3
1.5	Two popular transports	4
1.6	Demultiplexing packets	4
1.6.1	Ports	4
1.6.2	Connection lookup	5
2	Module 3.2 — UDP: User Datagram Protocol	6
2.1	Modularity through layering (recap)	6
2.2	UDP: best-effort service [RFC 768]	6
2.2.1	Why are UDP's (non-)guarantees even okay?	7
2.3	UDP segment structure	8
2.4	UDP inside the IP packet	9
2.5	Try it: capture UDP in Wireshark	9
3	Module 3.3 — UDP Error Detection: The Checksum	10
3.1	UDP: best-effort service (recap)	10
3.2	The key idea	10
3.3	Requirements of an error-detection function	10
3.4	The Internet checksum algorithm	11
3.4.1	1's complement addition	11
3.4.2	The slide's example, step by step	11
3.5	From the UDP specification (RFC 768)	12
3.6	Some observations on checksums	13
3.6.1	Errors the checksum misses	13
3.6.2	Checksums are insufficient for reliable delivery	13
3.7	Summary of UDP	14
4	Formula Sheet	14
5	Numerical Problems with Worked Solutions	14
6	Multiple-Choice Questions with Explanations	16

1 Module 3.1 — The Transport Layer

1.1 Where transport sits



1.2 Transport services and protocols

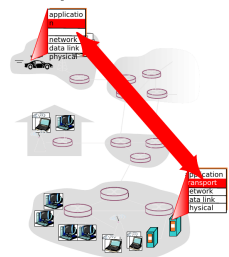
What the slide says (W3 L1, slide 3)

- Provide a **communication abstraction** between **application processes**.
- Transport protocols run @ **endpoints**.
- Send side: transport breaks app messages into **segments**, passes them to the network layer.
- Receive side: reassembles segments into messages, passes them to the app layer.
- Multiple transport protocols available to apps; very popular in the Internet: **TCP** and **UDP**.

From the lecture slides: W3 L1, slides 3–4 — transport services; transport vs network (household analogy)

Transport services and protocols

- Provide a **communication abstraction** between application processes
- Transport protocols run @ endpoints
 - send side: transport breaks app messages into **segments**, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer
- Multiple transport protocols available to apps
 - Very popular in the Internet: **TCP** and **UDP**

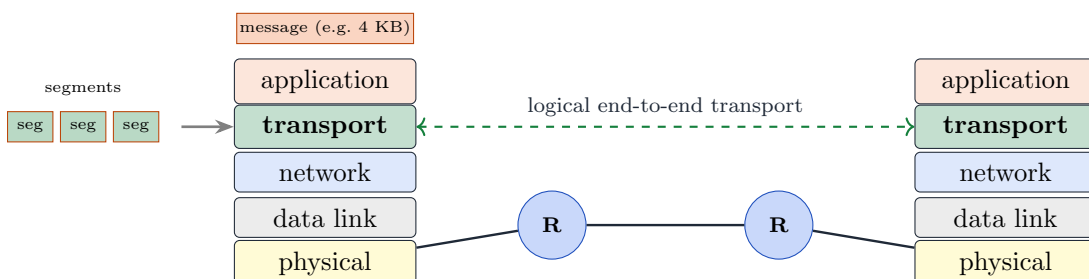


Transport vs. network layer

- **Network layer:** abstraction to communicate between **endpoints**. Network layer provides best effort packet delivery to a remote endpoint.
- **Transport layer:** communication abstraction between **processes**. Delivers packets to the process.

Household analogy:

- 12 kids sending letters to 12 kids
- processes = kids
 - app messages = letters in envelopes
 - endpoints = houses
 - transport protocol = Alice and Bob who de/mux to in-house siblings
 - network-layer protocol = postal service



Definition: Segment

The transport-layer unit of data: an application message chunk plus a transport header (ports, and for TCP: sequence numbers, ACKs, etc.). For UDP it is often called a *datagram*.

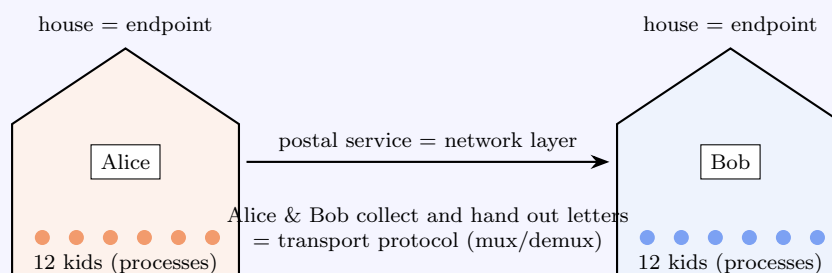
Intuition: Why the transport layer lives only at the ends

Routers in the middle operate only up to the network layer; they neither know nor care which process a packet is for. Only the two hosts need to know that “these bytes belong to the browser tab in process 4217”. This is the end-to-end principle again: put application-specific functionality at the endpoints, keep the core simple.

1.3 Transport vs network layer

	Network layer	Transport layer
Abstraction	communication between endpoints (hosts)	communication between processes
Service	best-effort packet delivery to a remote endpoint	delivers packets to the right process (+ optional guarantees)
Addressing	IP address	IP address + port
Where it runs	hosts <i>and</i> routers	hosts only

Intuition: The household analogy (slide)



Processes = kids; **app messages** = letters in envelopes; **endpoints** = houses; **transport protocol** = Alice and Bob, who multiplex/demultiplex letters to their in-house siblings; **network-layer protocol** = postal service. The postal service only gets letters to the right *house*; Alice and Bob get them to the right *kid*.

1.4 Identifying a single conversation: the 4-tuple

Definition: 4-tuple

An application connection is identified by (**source IP address, source port, destination IP address, destination port**). In the analogy: address of the first house, name of the kid there; address of the second house, name of the kid there.

From the lecture slides: W3 L1, slides 5 and 7 — the 4-tuple; TCP vs UDP

Identifying a single conversation

- Application connections are identified by 4-tuple:
- Source IP address
- Source port
- Destination IP address
- Destination port
- In this analogy,
- Source address: the address of the first house
- Source port: name of a kid in the first house
- Destination address: the address of the second house
- Destination port: name of a kid in the second house

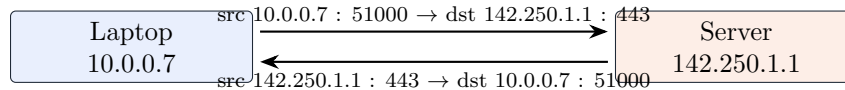
Two popular transports

Transmission Control Protocol (TCP)

- Connection-based: the application remembers the other process talking to it.
- Suitable for **longer-term, contextual data transfers**, like HTTP, file transfers, etc.
- Guarantees: reliability, ordering, congestion control

User Datagram Protocol (UDP)

- Connectionless: app doesn't remember the last process or source that talked to it.
- Suitable for **single req/resp flows**, like DNS.
- Guarantees: basic error detection



Notice that the reply swaps source and destination. Two browser tabs to the same server differ only in the client's source port — that's enough to keep the conversations separate.

1.5 Two popular transports

	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-based: the application remembers the other process talking to it	Connectionless: app doesn't remember the last process or source that talked to it
Suitable for	longer-term, contextual data transfers — HTTP, file transfers	single request/response flows — DNS
Guarantees	reliability, ordering, congestion control	basic error detection only
Socket type	<code>SOCK_STREAM</code> (byte stream)	<code>SOCK_DGRAM</code> (messages)
Header	20–60 bytes	8 bytes

Beyond the slides: Typical applications and their transports

Application	Transport	Why
Web (HTTP/1.1, HTTP/2), email (SMTP), file transfer, SSH	TCP	every byte must arrive, in order
DNS, DHCP, SNMP	UDP	tiny request/response; a setup handshake would double latency
Voice/video calls, live gaming	UDP (RTP)	a late packet is useless; don't wait for retransmits
HTTP/3 (QUIC)	UDP	builds its own reliability in user space, avoiding TCP's limits

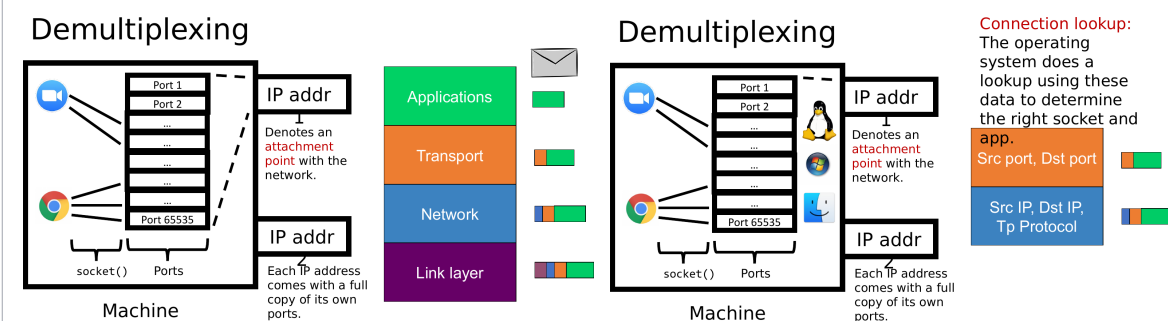
1.6 Demultiplexing packets

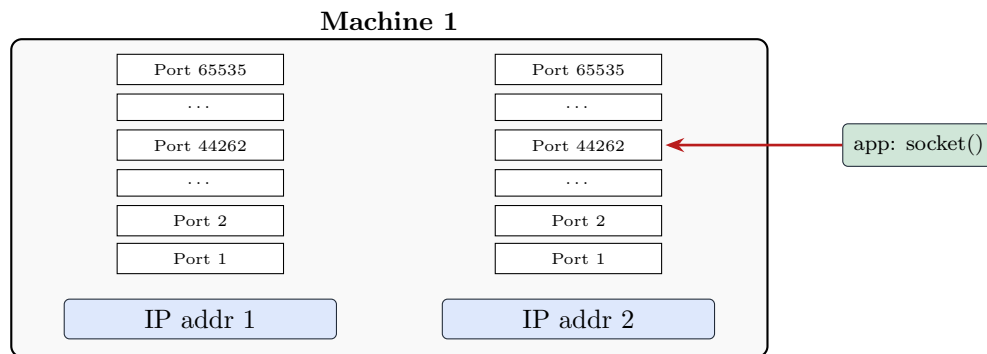
1.6.1 Ports

Definition: Port

A 16-bit number (0–65535) that denotes an **attachment point** of an application process with the network. A process obtains one by creating a `socket()` and binding it to a port. **Each IP address comes with a full copy of its own ports.**

From the lecture slides: W3 L1, slides 8 and 11 — ports as attachment points; header fields used for lookup





two IP addresses (e.g. WiFi + Ethernet) $\Rightarrow$ two independent sets of 65,536 ports

Beyond the slides: Port ranges

Range	Name	Examples
0–1023	well-known (need root to bind on Linux)	22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS
1024–49151	registered	3306 MySQL, 5432 PostgreSQL, 8080 alt-HTTP
49152–65535	dynamic / ephemeral	chosen by the OS for client sockets

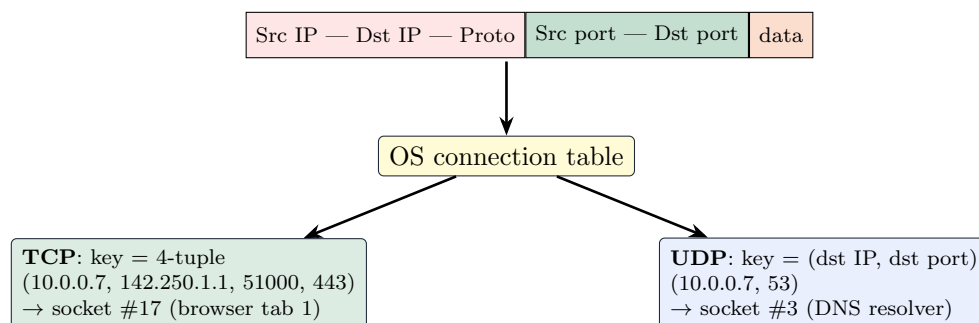
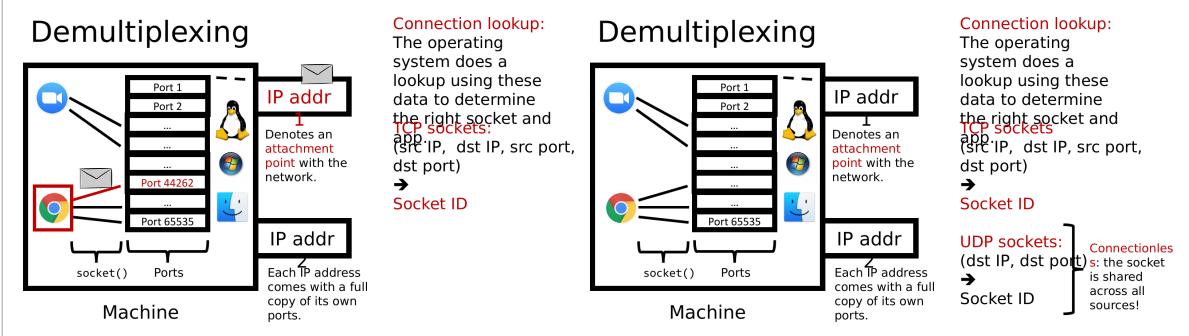
(Linux's default ephemeral range is 32768–60999 — the slide's example 44262 is one.)

1.6.2 Connection lookup

What the slide says (W3 L1, slides 10–15)

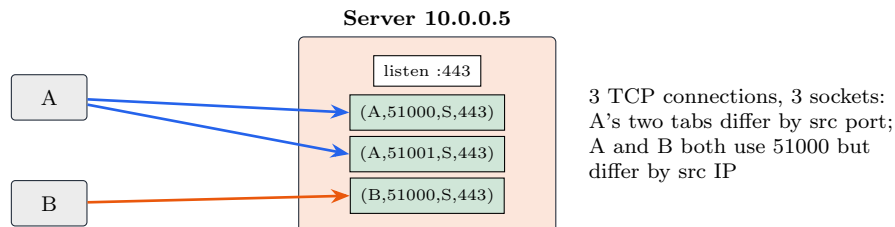
Header fields used: **Src IP, Dst IP, transport protocol, Src port, Dst port**. **Connection lookup**: the operating system does a lookup using these data to determine the right socket and app. **TCP sockets**: (src IP, dst IP, src port, dst port) $\Rightarrow$ socket ID. **UDP sockets**: (dst IP, dst port) $\Rightarrow$ socket ID.

From the lecture slides: W3 L1, slides 13 and 15 — TCP lookup by 4-tuple; UDP by (dst IP, dst port): one socket shared by all sources

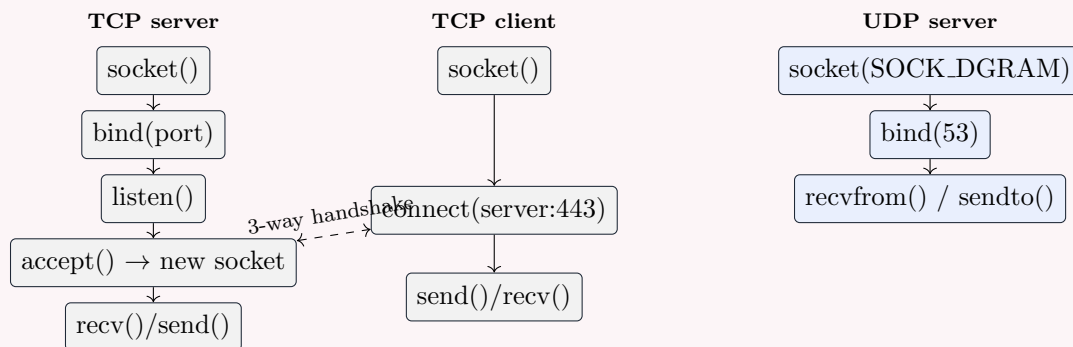


Intuition: Why TCP needs the full 4-tuple but UDP only needs 2 fields

A web server has *one* listening port (443) but thousands of simultaneous clients. Each TCP connection has its own state (sequence numbers, buffers), so the OS must route each segment to the right *connection* — only the full 4-tuple is unique per connection. UDP keeps no per-sender state: every datagram for port 53 goes into the *same* socket, and the app learns the sender from the `recvfrom()` result if it cares.



Beyond the slides: The socket API flow



UDP has no `listen/accept/connect`: there is no connection. `accept()` creating a *new* socket per client is exactly what the 4-tuple lookup demultiplexes to. (You'll write this in Project P2.)

2 Module 3.2 — UDP: User Datagram Protocol

2.1 Modularity through layering (recap)

What the slide says (W3 L2, slide 3)

Link: best-effort local packet delivery. Network: best-effort global packet delivery. Transport: provide guarantees to apps. Apps: useful user-level functions. (Protocol graph: HTTP, FTP, TFTP, NV over TCP/UDP over IP over Ethernet/ATM/WiFi.)

2.2 UDP: best-effort service [RFC 768]

What the slide says (W3 L2, slide 4)

Best effort service: UDP segments may be **lost** or **delivered out of order** to the app. UDP is **connectionless**: each UDP segment handled independently of others (no “memory” across packets). Suitable for one-off req/resp (e.g. DNS) and for loss-tolerant, delay-sensitive apps (e.g. video calling).

From the lecture slides: W3 L2, slide 4 — UDP (RFC 768) and why its guarantees are okay

UDP: User Datagram Protocol [RFC 768]

- **Best effort service.** UDP segments may be:
 - Lost
 - Delivered out of order to app
- UDP is **connectionless**
 - Each UDP segment handled **independently** of others (i.e. no “memory” across packets)
- Suitable for one-off req/resp
 - E.g., DNS uses UDP
- Also for loss-tolerant delay-sensitive apps, e.g., video calling

Why are UDP's guarantees even okay?

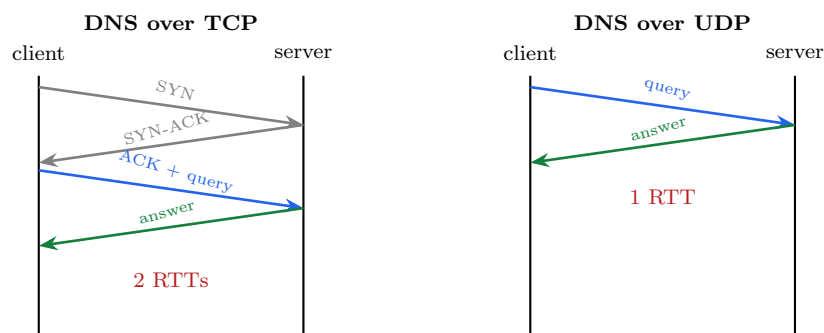
Simple & low overhead compared to TCP:

- No delays due to connection establishment
 - UDP can send data immediately
- No memory for connection state at sender & receiver
- Small segment header
- UDP can blast away data as fast as desired
 - UDP has no “congestion control”

2.2.1 Why are UDP's (non-)guarantees even okay?

Because UDP is **simple and low overhead** compared to TCP:

1. **No delays due to connection establishment** — UDP can send data immediately (TCP spends 1 RTT on a handshake).
2. **No memory for connection state** at sender and receiver — a DNS server can serve millions of clients without per-client state.
3. **Small segment header** — 8 bytes vs TCP's 20+.
4. **UDP can blast away data as fast as desired** — it has no congestion control that would slow it down.



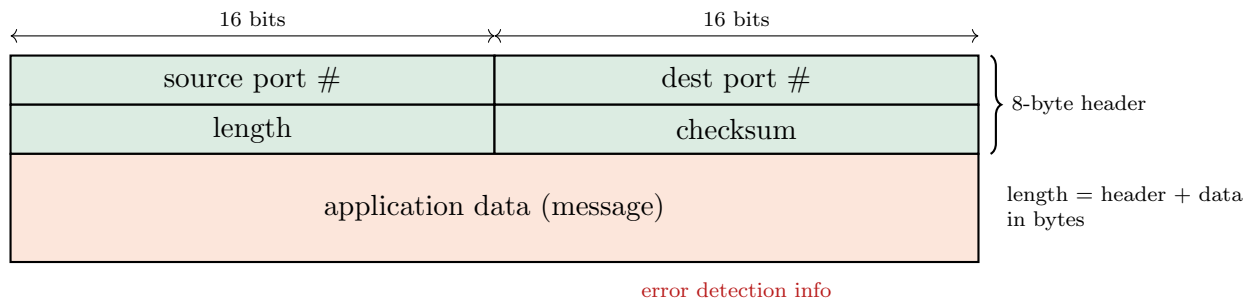
Intuition: Who handles loss then?

The application. A DNS resolver simply re-sends the query if no answer comes within a timeout (a tiny app-level reliability protocol). A video call just skips a lost frame — retransmitting it would arrive too late to be useful anyway. UDP lets the app choose its own trade-off instead of having TCP's choices forced on it.

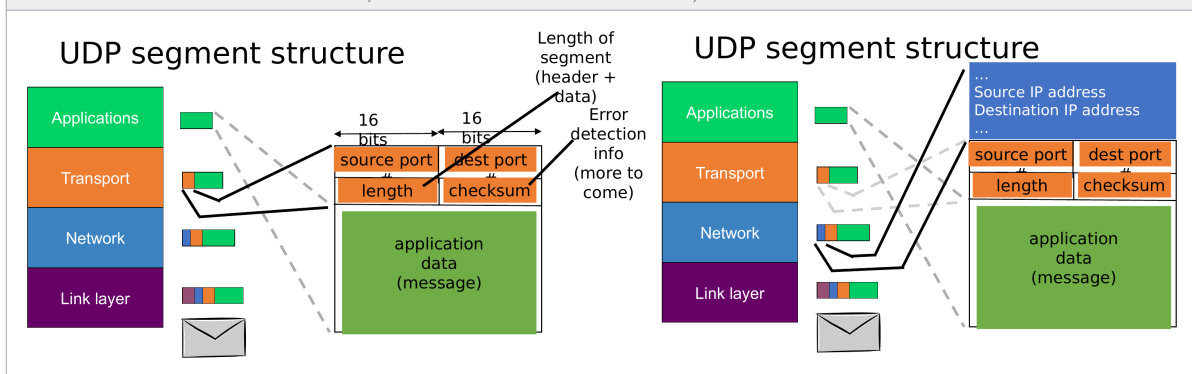
Common pitfall: UDP and the network

“Blast away as fast as desired” is a feature for the app but a risk for the network: a UDP flow that ignores congestion can starve TCP flows sharing a bottleneck. Well-behaved UDP apps (e.g. video conferencing, QUIC) implement their own congestion control.

2.3 UDP segment structure



From the lecture slides: W3 L2, slides 5–6 — UDP header fields; IP header carries the addresses



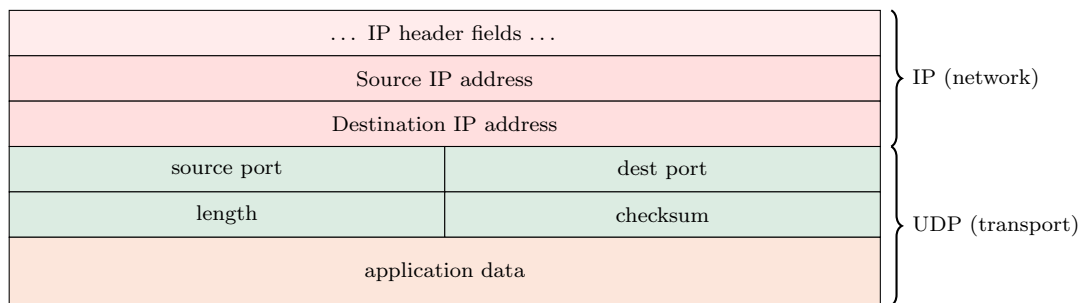
Field	Size	Meaning
Source port	16 bits	sender's port (where replies go); may be 0 if unused
Destination port	16 bits	receiving process's port — the demultiplexing key
Length	16 bits	length of the <i>whole segment</i> (header + data) in bytes; minimum 8
Checksum	16 bits	error detection over pseudo-header + header + data

Formula: UDP length limits

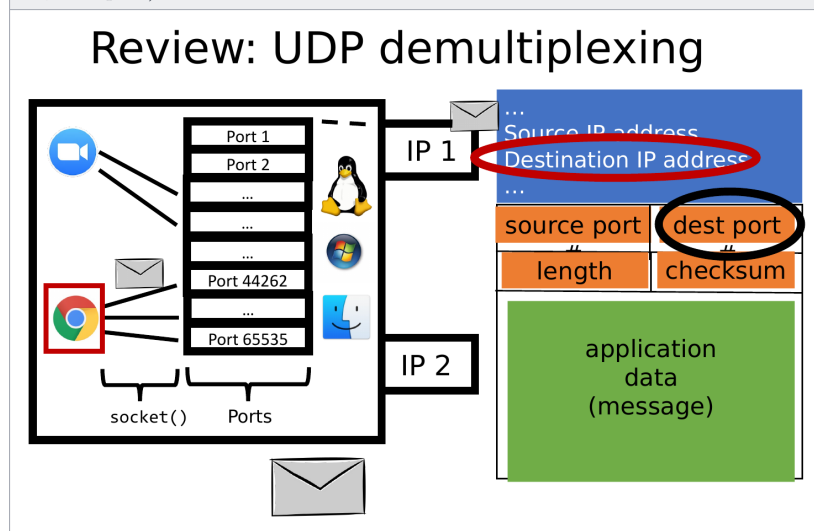
$$\text{Length} = 8 + |\text{data}| \quad [\text{bytes}]$$

16-bit field $\Rightarrow \max 2^{16} - 1 = 65,535$ bytes $\Rightarrow \max \text{data} = 65,527$ bytes. Over IPv4 (20-byte header, total length also 16-bit): $\max \text{data} = 65,535 - 20 - 8 = 65,507$ bytes.

2.4 UDP inside the IP packet



From the lecture slides: W3 L2, slide 7 — UDP demultiplexing on (destination IP, dest port)



UDP demultiplexing uses **(Destination IP address, dest port)** — the IP address comes from the network-layer header, the port from the UDP header. The source IP and port are passed up to the app so it can reply.

2.5 Try it: capture UDP in Wireshark

What the slide says (W3 L2, slide 8)

Try capturing UDP packets and analyse them in Wireshark.

Beyond the slides: A DNS query, byte by byte

Run `dig iitm.ac.in` while capturing with the filter `udp.port == 53`. A typical UDP header in hex:

Bytes	Hex	Field	Decoded
0--1	c8 3e	source port	51262 (ephemeral)
2--3	00 35	dest port	53 (DNS)
4--5	00 30	length	48 bytes (8 header + 40 DNS query)
6--7	7a 1f	checksum	verified by receiver

Good practice for Project P1: write a parser that decodes these four fields from raw bytes (big-endian, “network byte order”).

3 Module 3.3 — UDP Error Detection: The Checksum

3.1 UDP: best-effort service (recap)

What the slide says (W3 L3, slide 2)

Simple and low-overhead transport: connectionless. Data may be lost; data may be corrupted along the way (e.g. $1 \rightarrow 0$); data may be reordered. However, **simple error detection is possible**.

3.2 The key idea

What the slide says (W3 L3, slide 3)

Have the sender compute a function over the data; store the result in the packet; the receiver can check the function's value in the received packet.

From the lecture slides: W3 L3, slides 3–4 — the weighed-package idea; requirements of the function

UDP Error Detection

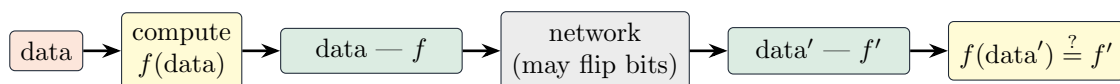
- Key idea: have sender compute a function over the data
 - Store the result in the packet
 - Receiver can check the function's value in received packet
- An analogy: you're sending a package of goodies and want your recipient to know if goodies were leaked along the way
- Your idea: weigh the package; stamp the weight on the package
 - Have the recipient weigh the package and cross-check the weight with the stamped value

Error detection function

- Function must be **easy to compute**
- Function must **capture the likely changes** to the packet
 - If the packet was corrupted through these likely changes, the function value must change
- Function must be **easy to verify**
- UDP uses a function called a **checksum**
 - Very common idea: used in multiple parts of networks and computer systems

Intuition: The package analogy

You're sending a box of goodies and want the recipient to know if goodies leaked along the way. Idea: weigh the package and stamp the weight on it. The recipient weighs it again and cross-checks with the stamp. Mismatch $\Rightarrow$ something happened.



3.3 Requirements of an error-detection function

1. **Easy to compute** — runs on every packet at line rate.
2. **Captures the likely changes** to the packet — if the packet was corrupted through likely changes (bit flips), the function value must change.
3. **Easy to verify** at the receiver.

UDP uses a function called a **checksum**. Very common idea: used in multiple parts of networks and computer systems (IP header, TCP, UDP; and in storage, file downloads, etc.).

3.4 The Internet checksum algorithm

Definition: Sender

Treat the segment contents as a sequence of **16-bit integers**. Checksum = addition (**1's complement sum**) of the segment contents, then complemented. Put the value into the UDP checksum field.

Definition: Receiver

Compute the checksum of the received segment, **including the checksum in the packet itself**. Check whether the resulting (computed) checksum is 0: **NO** — an error is detected; **YES** — assume no error.

Common pitfall: Two equivalent ways to state the receiver test

Sum all 16-bit words including the checksum field with 1's-complement addition. If there is no error, the *sum* is 1111 1111 1111 1111; its complement (the “computed checksum”) is 0. Exam questions use either phrasing.

3.4.1 1's complement addition

Definition: 1's complement sum

Very similar to regular (unsigned) binary addition, but when adding numbers, a **carry-out from the most significant bit is added back to the result** (“wraparound” / end-around carry).

3.4.2 The slide's example, step by step

1110 0110 0110 0110	
+ 1101 0101 0101 0101	
1 1011 1011 1011 1011	17-bit result: carry out of the MSB
1011 1011 1011 1011	
+ 1	wraparound: add the carry to the LSB
1011 1011 1011 1100	sum
0100 0100 0100 0011	checksum = flip every bit of the sum

From the lecture slides: W3 L3, slides 5–6 — sender/receiver rules and the 1's-complement example

UDP Checksum

Sender:

- treat segment contents as sequence of 16-bit integers
- checksum: addition (**1's complement sum**) of segment contents
- sender puts checksum value into **UDP checksum** field

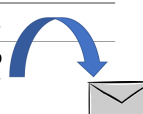
Receiver:

- compute a checksum of the received segment, **including the checksum in packet itself**
- check if the resulting (computed) checksum is 0
- **NO** – an error is detected
- YES – assume no error

Computing 1's complement sum

- Very similar to regular (unsigned) binary addition.
- However, when adding numbers, a carryout from the most significant bit needs to be added to the result
- Example: add two 16-bit integers

	1110011001100110
	1101010101010101
wraparound	① 1011101110111011
sum	1011101110111100
checksum	0100010001000011



6

Receiver check. 1011 1011 1011 1100 + 0100 0100 0100 0011 = 1111 1111 1111 1111 (no carry) $\Rightarrow$

complement = 0 $\Rightarrow$ no error detected. Adding a number and its bitwise complement always gives all 1s — that’s why the test works.

Beyond the slides: Algorithm (pseudocode)

```

function checksum(bytes):
  if len(bytes) is odd: append one zero byte
  sum  $\leftarrow$  0
  for each pair of bytes (hi, lo):
    word  $\leftarrow$  hi  $\times$  256 + lo      (big-endian 16-bit word)
    sum  $\leftarrow$  sum + word
    sum  $\leftarrow$  (sum AND 0xFFFF) + (sum SHIFT-RIGHT 16)  (wraparound)
  return (NOT sum) AND 0xFFFF

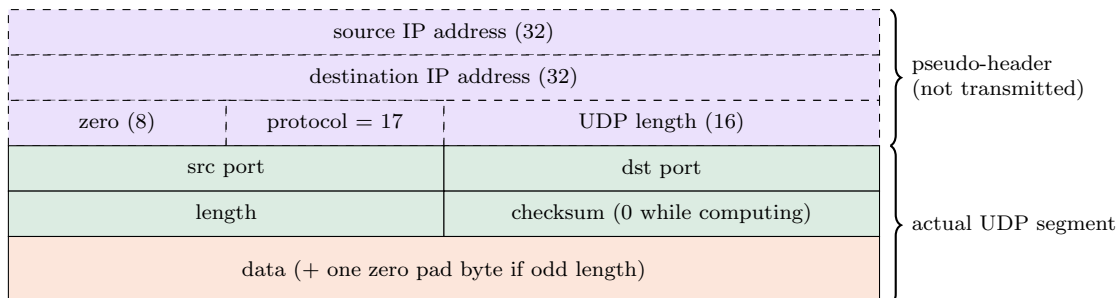
```

Receiver: run the same loop over everything including the checksum field; accept iff the final sum is 0xFFFF.

3.5 From the UDP specification (RFC 768)

What the slide says (W3 L3, slide 7)

Checksum is the 16-bit one’s complement of the one’s complement sum of a **pseudo header** of information from the **IP header**, the **UDP header**, and **the data**, padded with zero octets at the end (if necessary) to make a multiple of two octets. The pseudo header conceptually prefixed to the UDP header contains the **source address**, the **destination address**, the **protocol**, and the **UDP length**.



Intuition: Why include IP addresses in a transport checksum?

If a bit error in the IP header changed the destination address, the packet could be delivered to the wrong host, and a checksum over only the UDP part would still pass. Including the addresses (the pseudo-header) means a mis-delivered segment fails the check. The pseudo-header is never sent — both sides reconstruct it from the IP header. (This is a deliberate crossing of layer boundaries.)

Beyond the slides: Details worth knowing

- While computing, the checksum field itself is set to 0.
- In IPv4, UDP checksum is optional: a transmitted value of 0 means “not computed”. (A computed value of 0 is sent as 0xFFFF, which is the same number in 1’s complement arithmetic.) In IPv6 it is mandatory.
- Order doesn’t matter: 1’s complement addition is commutative and associative, so words can be summed in any order — and byte order (endianness) of the machine doesn’t change the result.

- Incremental update: if a router changes one 16-bit field (e.g. NAT rewriting a port), it can fix the checksum without recomputing over the whole packet.

3.6 Some observations on checksums

What the slide says (W3 L3, slide 8)

Checksums don't detect all bit errors: consider (x, y) vs $(x-1, y+1)$ as adjacent 16-bit values in a packet. Analogy: you can't assume the package hasn't been meddled with if its weight matches the stamp. But it's a lightweight method that works well in many cases. Checksums are part of the packet; they can get corrupted too — the receiver just declares an error. Checksums don't let the receiver locate or correct errors: **detection, not correction**.

From the lecture slides: W3 L3, slides 7–9 — RFC 768 text; observations on checksums

From the UDP specification (RFC 768)

- Checksum is the 16-bit one's complement of the one's complement sum of a pseudo header of information from the IP header, the UDP header, and the data, padded with zero octets at the end (if necessary) to make a multiple of two octets.
- The pseudo header conceptually prefixed to the UDP header contains the source address, the destination address, the protocol, and the UDP length.

Some observations on checksums

- Checksums don't detect all bit errors
 - Consider (x, y) vs. $(x - 1, y + 1)$ as adjacent 16-bit values in packet
 - Analogy: you can't assume the package hasn't been meddled with if its weight matches the one on the stamp. More smarts needed for that. ☹
 - But it's a lightweight method that works well in many cases
- Checksums are part of the packet; they can get corrupted too
 - The receiver will just declare an error if it finds an error
 - However, checksums don't enable the receiver to detect where the error lies or correct the error(s)
 - Checksum is an error detection mechanism; not a correction mechanism.

Some observations on checksums

- Checksums are insufficient for reliable data delivery
 - If a packet is lost, so is its checksum
- UDP and TCP use the same checksum function
 - TCP also uses the lightweight error detection capability
 - However, TCP has more mature mechanisms for generally reliable data delivery (lots more to come on this)

3.6.1 Errors the checksum misses

Corruption	Example	Detected?
Any single bit flip	one bit changes	Yes, always
Compensating changes	$(x, y) \rightarrow (x-1, y+1)$	No — sum unchanged
Two flips of the same bit position in two words (0→1 in one, 1→0 in other)	same column	No
Reordering 16-bit words	swap w_1, w_2	No — addition is commutative
Inserting/removing all-zero words	extra 0x0000	No

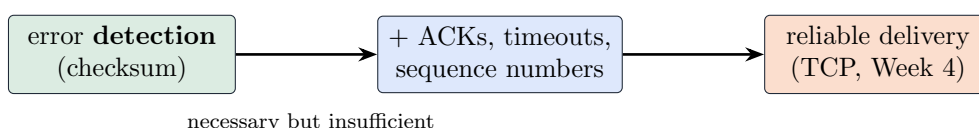
Intuition

The checksum is just a *sum*: anything that preserves the sum slips through — like swapping two items of equal weight in the box. Stronger codes (CRC at the link layer, cryptographic hashes for file downloads) catch far more, at higher cost. The Internet relies on link-layer CRCs to catch most errors and the transport checksum as an end-to-end backstop.

3.6.2 Checksums are insufficient for reliable delivery

What the slide says (W3 L3, slide 9)

If a packet is lost, so is its checksum. UDP and TCP use the **same** checksum function; TCP also uses this lightweight error-detection capability. However, TCP has more mature mechanisms for generally reliable data delivery (lots more to come).



3.7 Summary of UDP

Summary: UDP (slide)

- UDP is a **thin shim** around the network layer's best-effort delivery.
- Used for one-off request/response messages, and as a lightweight transport for loss-tolerant, delay-sensitive applications.
- Provides basic **multiplexing/demultiplexing** for applications.
- **No** reliability, performance, or ordering guarantees.
- Can do basic **error detection** (bit flips) using checksums.
- Error detection is necessary to deliver data reliably, but it is insufficient.

4 Formula Sheet

TCP demux key	(src IP, dst IP, src port, dst port) — plus protocol
UDP demux key	(dst IP, dst port)
Port field	16 bits: 0–65535; well-known 0–1023
UDP header	8 bytes: src port, dst port, length, checksum (16 bits each)
UDP length	8 + data bytes; max data 65,527 (UDP), 65,507 (over IPv4)
1's complement add	normal binary add; carry out of bit 15 is added back at bit 0
Checksum	$\text{NOT}(\bigoplus_{1's} \text{16-bit words of pseudo-hdr} + \text{hdr} + \text{data})$
Receiver check	1's-comp sum of all words incl. checksum = 0xFFFF (its complement = 0) $\Rightarrow$ accept
Pseudo-header	src IP, dst IP, zero, protocol (17), UDP length — 12 bytes, not transmitted

5 Numerical Problems with Worked Solutions

Problem 1. Checksum — the slide's example

Compute the checksum of the two 16-bit words 1110 0110 0110 0110 and 1101 0101 0101 0101, and verify it.

Solution

Sum = 1 1011 1011 1011 1011; wrap the carry: 1011 1011 1011 1100. Checksum = complement = 0100 0100 0100 0011. Verification: sum + checksum = 1111 1111 1111 1111 $\Rightarrow$ OK.

Problem 2. Checksum of three words

Compute the checksum of 0110 0110 0110 0000, 0101 0101 0101 0101, 1000 1111 0000 1100.

Solution

0110011001100000 + 0101010101010101 = 1011101110110101 (no carry).
 1011101110110101 + 1000111100001100 = 10100101011000001; wrap:
 0100101011000010.

Checksum = 1011 0101 0011 1101. Check: 0100101011000010 + 1011010100111101 = 1111111111111111 ✓

Problem 3. *Checksum in hex*

Compute the checksum of the words 0x4500, 0x0030, 0x4422, 0x4000.

Solution

0x4500 + 0x0030 = 0x4530; + 0x4422 = 0x8952; + 0x4000 = 0xC952 (no carries). Checksum = NOT(0xC952) = 0xFFFF − 0xC952 = 0x36AD. Check: 0xC952 + 0x36AD = 0xFFFF ✓. (Trick: in hex, the complement of each digit d is $15 - d$.)

Problem 4. *Hex with a carry*

Compute the checksum of 0xF0F0 and 0x1234.

Solution

0xF0F0 + 0x1234 = 0x10324 (17 bits). Wrap: 0x0324 + 0x1 = 0x0325. Checksum = 0xFFFF − 0x0325 = 0xFCDA. Check: 0x0325 + 0xFCDA = 0xFFFF ✓.

Problem 5. *Receiver verification — error present*

A receiver gets data words 1110 0110 0110 0110, 1101 0101 0101 0101 and checksum 0100 0100 0100 0010. Is an error detected?

Solution

Sum of data = 1011 1011 1011 1100 (Problem 1). Add the received checksum: 1011101110111100 + 0100010001000010 = 1111111111111110. Not all 1s (complement = 0000 0000 0000 0001 $\neq$ 0) $\Rightarrow$ **error detected**.

Problem 6. *An error the checksum misses*

Words 0x1234, 0x0001 are corrupted in transit to 0x1233, 0x0002. Is it detected?

Solution

Original sum 0x1235; corrupted sum 0x1233 + 0x0002 = 0x1235. Identical sums $\Rightarrow$ identical checksums $\Rightarrow$ **not detected** (the slide's (x−1, y+1) case).

Problem 7. *Odd-length data*

The UDP data is the 3 bytes 0xAB 0xCD 0xEF. What 16-bit words enter the checksum (ignore headers)?

Solution

Pad with one zero byte at the end: words 0xABCD and 0xEF00. The pad byte is not transmitted and doesn't count in the length field.

Problem 8. *UDP length and overhead*

(a) A DNS query carries 40 bytes of data. Find the UDP length field and the IPv4 datagram size (20-byte IP header). (b) A VoIP app sends 160-byte voice payloads every 20 ms. Find the UDP length, the header overhead (UDP + IP) as a fraction of the IP datagram, and the bit rate at the IP layer.

Solution

- (a) UDP length = $8 + 40 = 48$ bytes; IP datagram = $20 + 48 = 68$ bytes.
 (b) UDP length = 168 B; IP datagram = 188 B. Overhead = $28/188 = 14.9\%$. Rate = $188 \times 8/0.02 = 75.2$ kbps (vs 64 kbps of pure voice).

Problem 9. *Maximum UDP payload*

What is the largest application message that fits in one UDP datagram over IPv4? Over IPv4 with a 1500-byte Ethernet MTU without fragmentation?

Solution

IPv4 total-length field max 65,535 $\Rightarrow 65,535 - 20 - 8 = 65,507$ bytes. Without fragmentation on Ethernet: $1500 - 20 - 8 = 1472$ bytes.

Problem 10. *Demultiplexing*

Server 10.0.0.5 runs a web server on TCP 443 and DNS on UDP 53. Clients: 10.0.0.7:51000, 10.0.0.7:51001 and 10.0.0.9:51000 open HTTPS connections; 10.0.0.7:40000 and 10.0.0.9:40000 send DNS queries. How many sockets does the server use, and what keys identify them?

Solution

TCP: 1 listening socket on 443, plus one connected socket per 4-tuple: (10.0.0.7, 51000, 10.0.0.5, 443), (10.0.0.7, 51001, 10.0.0.5, 443), (10.0.0.9, 51000, 10.0.0.5, 443) $\Rightarrow 3$. UDP: both queries go to the **same** socket keyed by (10.0.0.5, 53). Total = $1 + 3 + 1 = 5$ sockets.

Problem 11. *Setup cost: TCP vs UDP*

A client makes 1000 independent DNS-style lookups, each a single small request and reply, to a server with RTT 30 ms. Compare total time if each lookup uses (a) UDP, (b) a fresh TCP connection (1 RTT handshake, then request/response; ignore teardown).

Solution

- (a) 1 RTT each: $1000 \times 30 = 30$ s. (b) 2 RTTs each: 60 s. UDP halves latency for one-shot exchanges — the slide's “no delays due to connection establishment”.

6 Multiple-Choice Questions with Explanations

Q1. The transport layer provides a communication abstraction between:

- (a) Hosts
- (b) Processes
- (c) Routers
- (d) Links

(b). Network layer connects hosts.

Q2. Transport protocols run:

- (a) In every router
- (b) Only at endpoints
- (c) Only in switches
- (d) In DNS servers only

(b).

Q3. On the send side, the transport layer:

- (a) Adds a MAC address
- (b) Breaks app messages into segments
- (c) Chooses the route
- (d) Encodes bits as signals

(b).

Q4. In the household analogy, the transport protocol corresponds to:

- (a) The postal service
- (b) The houses
- (c) Alice and Bob, who distribute letters to siblings
- (d) The letters

(c). Postal service = network layer.

Q5. A connection is identified by:

- (a) Source IP only
- (b) (src IP, src port, dst IP, dst port)
- (c) Destination port only
- (d) MAC addresses

(b).

Q6. Port numbers are:

- (a) 8 bits
- (b) 16 bits
- (c) 32 bits
- (d) 48 bits

(b). 0–65535.

Q7. “Each IP address comes with a full copy of its own ports” means:

- (a) Ports are shared between all IPs
- (b) A host with 2 IPs has 2 independent port spaces
- (c) Ports are global across the Internet
- (d) Only one IP can use port 80

(b).

Q8. A TCP socket is looked up using:

- (a) (dst IP, dst port)
- (b) (src IP, dst IP, src port, dst port)
- (c) (src port)
- (d) (dst IP)

(b).

Q9. A UDP socket is looked up using:

- (a) The 4-tuple
- (b) (dst IP, dst port)
- (c) (src IP, src port)
- (d) Sequence number

(b).

Q10. Two browser tabs on the same laptop connected to the same server port are distinguished by:

- (a) Destination IP
- (b) Destination port
- (c) Source port
- (d) MAC address

(c).

Q11. TCP is described as:

- (a) Connectionless
- (b) Connection-based
- (c) Stateless
- (d) Unreliable

(b).

Q12. Which guarantee does UDP provide?

- (a) Reliability
- (b) Ordering
- (c) Congestion control
- (d) Basic error detection

(d).

Q13. DNS commonly uses UDP because:

- (a) DNS needs ordering
- (b) Single request/response; no handshake delay
- (c) UDP is reliable
- (d) TCP can't carry text

(b).

Q14. Which is *not* a reason UDP's lack of guarantees is acceptable?

- (a) No connection setup delay
- (b) No connection state
- (c) Small header
- (d) Guaranteed delivery

(d).

Q15. UDP segments may be:

- (a) Lost or reordered
- (b) Only delayed
- (c) Never corrupted

(d) Retransmitted automatically

(a).

Q16. The UDP header is:

- (a) 4 bytes
- (b) 8 bytes
- (c) 20 bytes
- (d) 40 bytes

(b).

Q17. UDP's length field covers:

- (a) Data only
- (b) Header only
- (c) Header + data
- (d) IP + UDP

(c).

Q18. Which UDP header field is used for demultiplexing?

- (a) Length
- (b) Checksum
- (c) Destination port
- (d) Source port

(c). Together with the destination IP.

Q19. UDP “can blast away data as fast as desired” because:

- (a) It has no congestion control
- (b) It uses compression
- (c) It has priority
- (d) Routers favour it

(a).

Q20. The Internet checksum treats data as a sequence of:

- (a) 8-bit bytes
- (b) 16-bit integers
- (c) 32-bit words
- (d) Bits

(b).

Q21. In 1's complement addition, a carry out of the MSB is:

- (a) Discarded
- (b) Added back to the result
- (c) Stored separately
- (d) An error

(b).

Q22. The checksum placed in the packet is:

- (a) The 1's complement sum
- (b) The complement of the 1's complement sum
- (c) The 2's complement
- (d) The CRC

(b).

Q23. If there is no error, the receiver's 1's complement sum over all words including the checksum is:

- (a) All 0s
- (b) All 1s
- (c) Equal to the checksum
- (d) Equal to the length

(b). Its complement is 0.

Q24. The UDP pseudo-header contains:

- (a) Ports and length
- (b) Source IP, destination IP, protocol, UDP length
- (c) MAC addresses
- (d) Sequence numbers

(b).

Q25. The pseudo-header is:

- (a) Transmitted before the UDP header
- (b) Reconstructed at both ends, not transmitted
- (c) Part of the IP header
- (d) Optional padding

(b).

Q26. Odd-length UDP data is handled by:

- (a) Rejecting it
- (b) Padding with a zero byte for the checksum
- (c) Adding a 1 byte
- (d) Truncating the last byte

(b).

Q27. Which error will the Internet checksum always detect?

- (a) Any single-bit flip
- (b) Word reordering
- (c) $(x-1, y+1)$
- (d) Inserted zero words

(a).

Q28. Checksums enable the receiver to:

- (a) Correct errors

- (b) Locate errors
- (c) Detect (some) errors
- (d) Recover lost packets

(c). Detection, not correction.

Q29. Why are checksums insufficient for reliability?

- (a) They are too slow
- (b) A lost packet loses its checksum too
- (c) They only work for TCP
- (d) They need encryption

(b).

Q30. UDP and TCP use:

- (a) Different checksum algorithms
- (b) The same checksum function
- (c) No checksum
- (d) CRC-32

(b).

Q31. UDP is best summarised as:

- (a) A reliable byte stream
- (b) A thin shim over best-effort IP with ports and a checksum
- (c) A routing protocol
- (d) A link-layer protocol

(b).